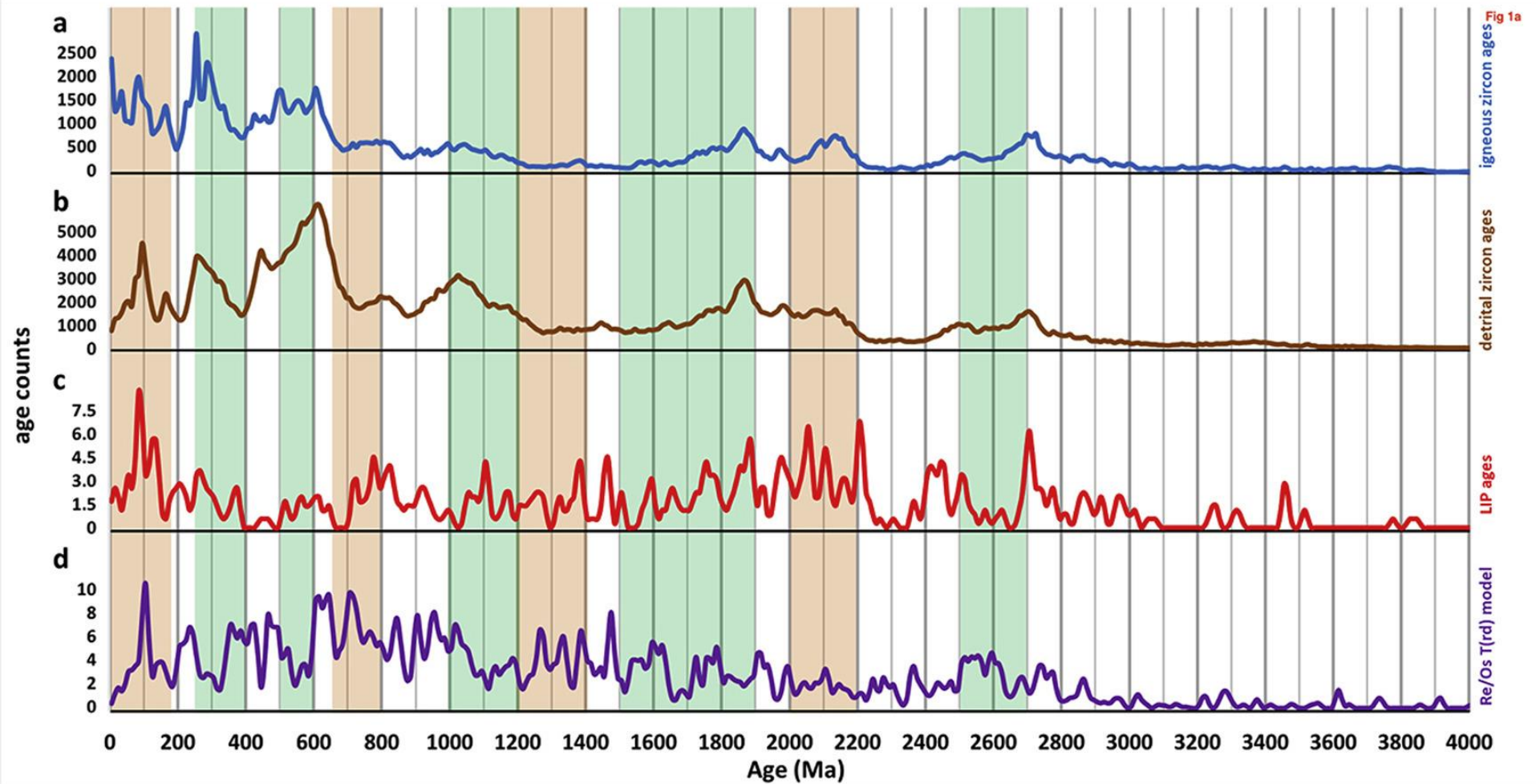


# Lecture 12 – Time Series II



# Today's Learning Outcomes

1. Be able to fit a moving average to a time series
2. Be able to explain the properties of a moving average
3. Be able to explain the concept of STL

# You've detected a Trend...now what?

A “trend” represents the relatively smooth “background” level of the time series, changing “slowly” over time.

Probably can be modeled as a smooth function

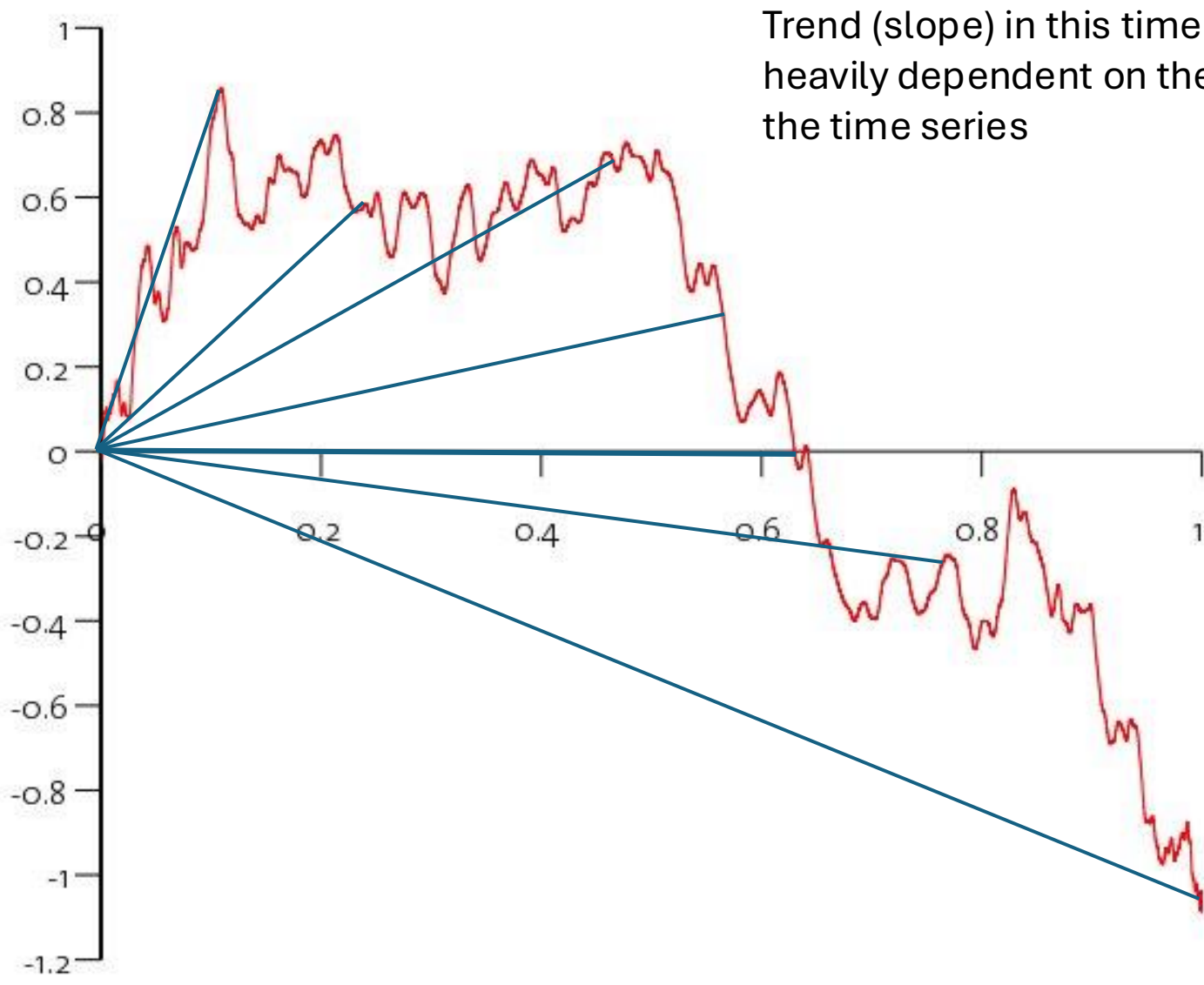
1. polynomials ( $x^0, x^1, x^2, ax^2+bx+c$ , etc.)
2. harmonic functions (sin, cos, tan)
3. piecewise linear functions (for  $x < 10$ ,  $y=x$ ; for  $x > 10$ ,  $y=x^2$ )

# Considerations When Modeling Trends

1. Asymptotic behavior over long time frames-polynomial models are never bounded
  - Values cannot increase forever in reality
2. Do we want to fit the same model to all part of the time series?
  - Dissolved Inorganic Carbon (DIC) in a stream pre- and post-construction of a dam upstream
3. Do we have an a-priori reason to chose a particular form of the model for the trend?

# Why Use Multiple Trend Functions?

- Single trend may be difficult to update when new observations become available
- Decent description of full time series may require many parameters, which are difficult to estimate
  - More data needed, remember AIC penalizes complexity
- As the model is updated, the entire set of trend values may be updated



Trend (slope) in this time series is heavily dependent on the length of the time series

# Modeling Trends

Attempts to find the “trend” via filtering

- Placing all the “slowly” changing components into a category (= trend) and all the “rapidly” changing terms into other categories (= periodicity & error)

1. How can this be done?

- SMA (**S**imple **M**oving **A**verage)
- LOWESS (**L**Ocally **W**Eighted regre**SS**ion)
- STL (**S**easonal-**T**rend **L**OESS)

2. How do we determine the effectiveness of these approaches?

# Simple Moving Averages

A rolling mean takes a fixed window and averages values equally:

$$\hat{y}_t = \frac{1}{k} \sum_{i=t-k+1}^t y_i$$

It is a **linear, non-adaptive** smoother.

## When it works well

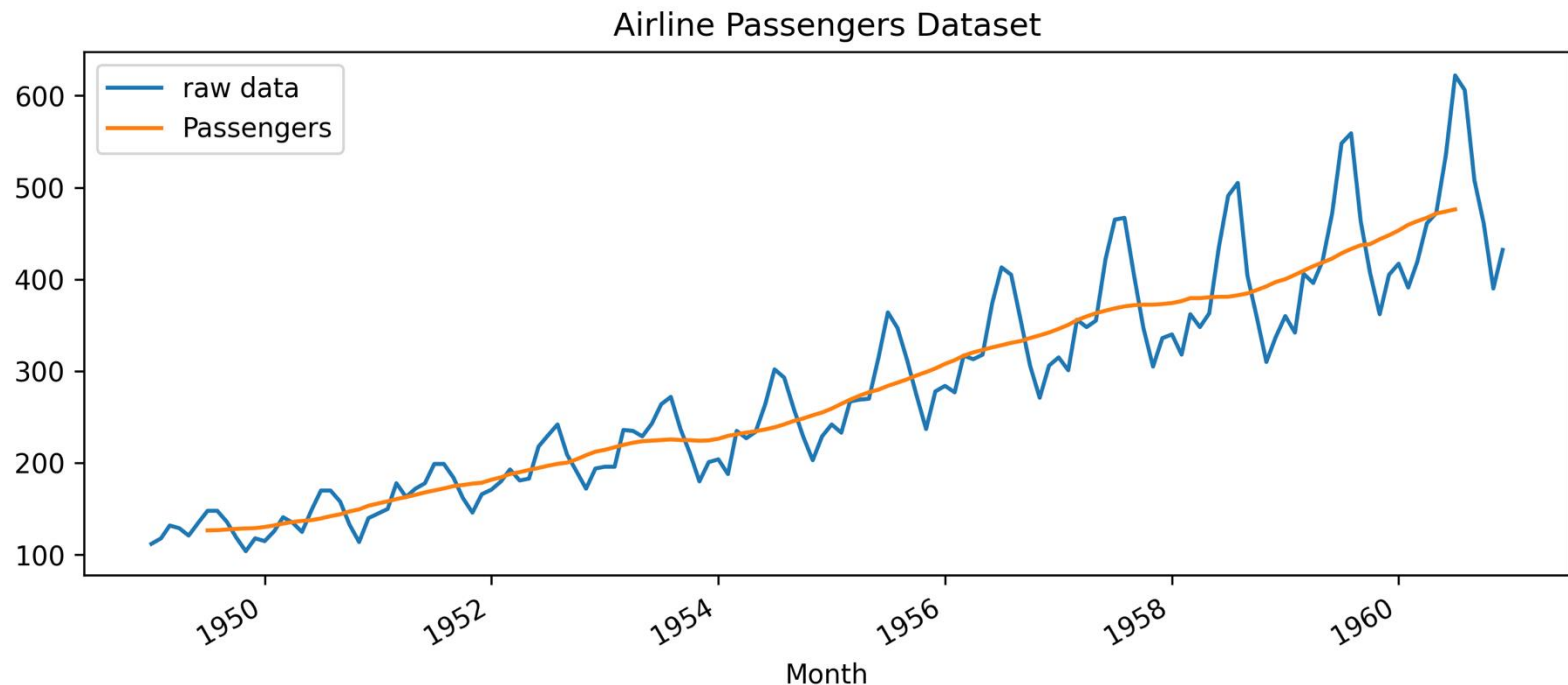
- Removing high-frequency noise
- Very smooth series without strong curvature
- Good for real-time / streaming applications

# Simple Moving Averages (SMA) in Python

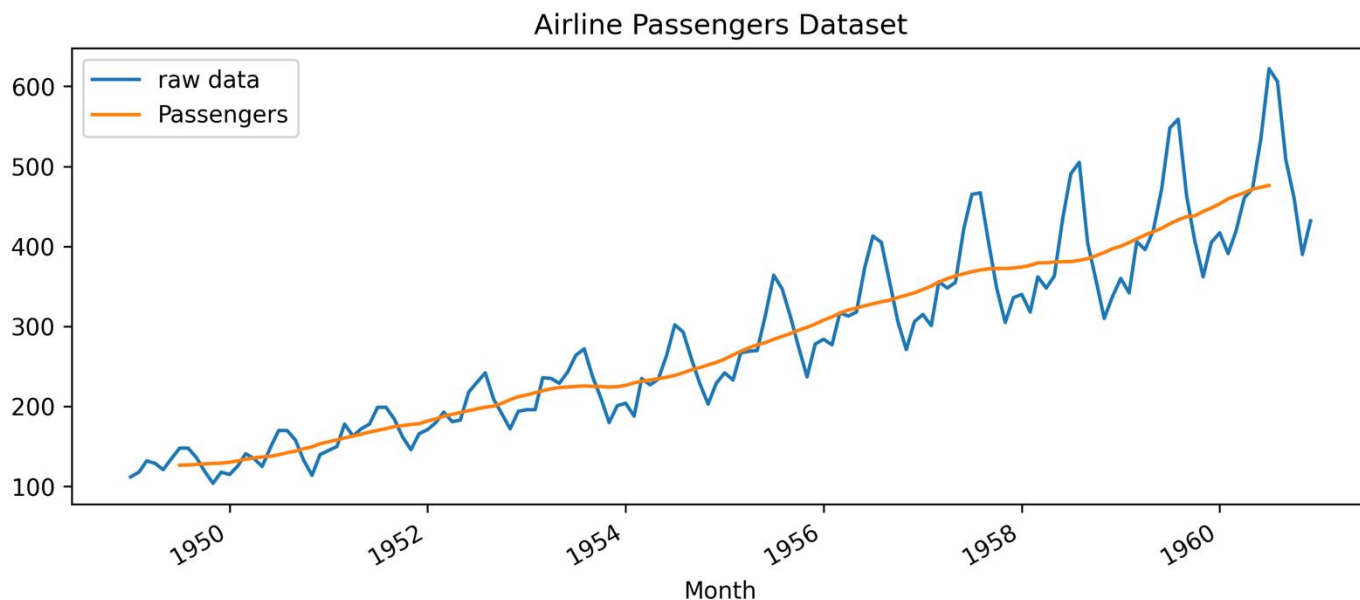
- Pandas and Numpy already have rolling mean functions embedded in them!
- `rolling(window=n)` , a function in Python returns the average over a window of width n (n= number of time steps)
  - ex. `df.rolling(window=12).mean()`

# SMA example

```
fig, ax = plt.subplots(1, 1, figsize=[10, 4], dpi=300)
ax.plot(data.index, data["Passengers"], label='raw data')
data.rolling(window=12, center=True).mean().plot(label='12 month rolling mean', ax=ax)
ax.set_title("Airline Passengers Dataset")
ax.legend()
```



# SMA Challenges



- Cannot model nonlinear trends
- Distorts seasonal cycles
- Requires *integer window sizes*
- Poor performance at edges

# Local Regression (LOESS)

Why move beyond the moving average?

- Moving averages are simple but:
  - Use **fixed window size**
  - Treat all points **equally**
  - Struggle with **trends, edges, and nonlinear shapes**
- LOESS solves these issues by:
  - Using **weights** instead of equal averages
  - Fitting **local regressions**
  - Producing smooth, adaptive curves

# Local Regression (LOESS)

LOESS in one sentence:

**Fit a small regression around each point, giving nearby points more influence.**

How it works conceptually:

- Pick a point you want to estimate
- Look at data around it
- Weight nearby points more than far ones
- Fit a simple model (usually linear)
- Repeat for every  $x$

**LOESS is like a *smart moving average* that adapts to the local pattern.**

# LOESS: Local Model + Weights = Smooth, Adaptive Fit

**Goal:** for each target  $x_0$ , fit a weighted local polynomial to predict  $y(x_0)$ .

**Choose the local polynomial degree  $d$**

- $d$  usually ranges from 0 to 3, and  $d=1$  (linear regression) is the most common one.
- $d = 1$  (local linear, most common): solve for  $\beta_0, \beta_1$  in

$$\min_{\beta_0, \beta_1} \sum_{i=1}^n w_i(x_0) [y_i - \beta_0 - \beta_1(x_i - x_0)]^2, \quad \hat{y}(x_0) = \hat{\beta}_0$$

# LOESS: Local Model + Weights = Smooth, Adaptive Fit

This slide is only for reference!

**Choose the local polynomial degree  $d$**

- $d = 0$  (local constant / Nadaraya–Watson):

$$\hat{y}(x_0) = \frac{\sum_{i=1}^n w_i(x_0) y_i}{\sum_{i=1}^n w_i(x_0)}$$

- $d = 1$  (local linear, most common): solve for  $\beta_0, \beta_1$  in

$$\min_{\beta_0, \beta_1} \sum_{i=1}^n w_i(x_0) [y_i - \beta_0 - \beta_1(x_i - x_0)]^2, \quad \hat{y}(x_0) = \hat{\beta}_0$$

- $d = 2$  (local quadratic, better curvature): solve for  $\beta_0, \beta_1, \beta_2$  in

$$\min_{\beta_0, \beta_1, \beta_2} \sum_{i=1}^n w_i(x_0) [y_i - \beta_0 - \beta_1(x_i - x_0) - \beta_2(x_i - x_0)^2]^2, \quad \hat{y}(x_0) = \hat{\beta}_0$$

*Centering at  $x_0$  improves numerical stability.*

# Weight function

Weights decrease with distance from  $x_0$  .

Typical choices:

- **Gaussian** (infinite support):

$$w_i(x_0) = \exp\left(-\frac{(x_i - x_0)^2}{2\tau^2}\right)$$

- **Tricube** (compact support; classic LOESS):

$$u_i = \frac{|x_i - x_0|}{h}, w_i(x_0) = \begin{cases} (1 - u_i^3)^3, & u_i < 1 \\ 0, & u_i \geq 1 \end{cases}$$

where  $h$  is the distance to the  $q$ -th nearest neighbor (the **span**).

## Smoothing control

- **Bandwidth / span** ( $\tau$  or  $h/q$ ): smaller  $\rightarrow$  more local (wigglier); larger  $\rightarrow$  smoother (risk of over-smoothing).
- **Degree**  $d$ : higher degree captures curvature but can overfit when span is small.

# LOESS in Python – synthetic data example

# 1) Create a synthetic seasonal time series

```
np.random.seed(7); N = 500; period = 24
```

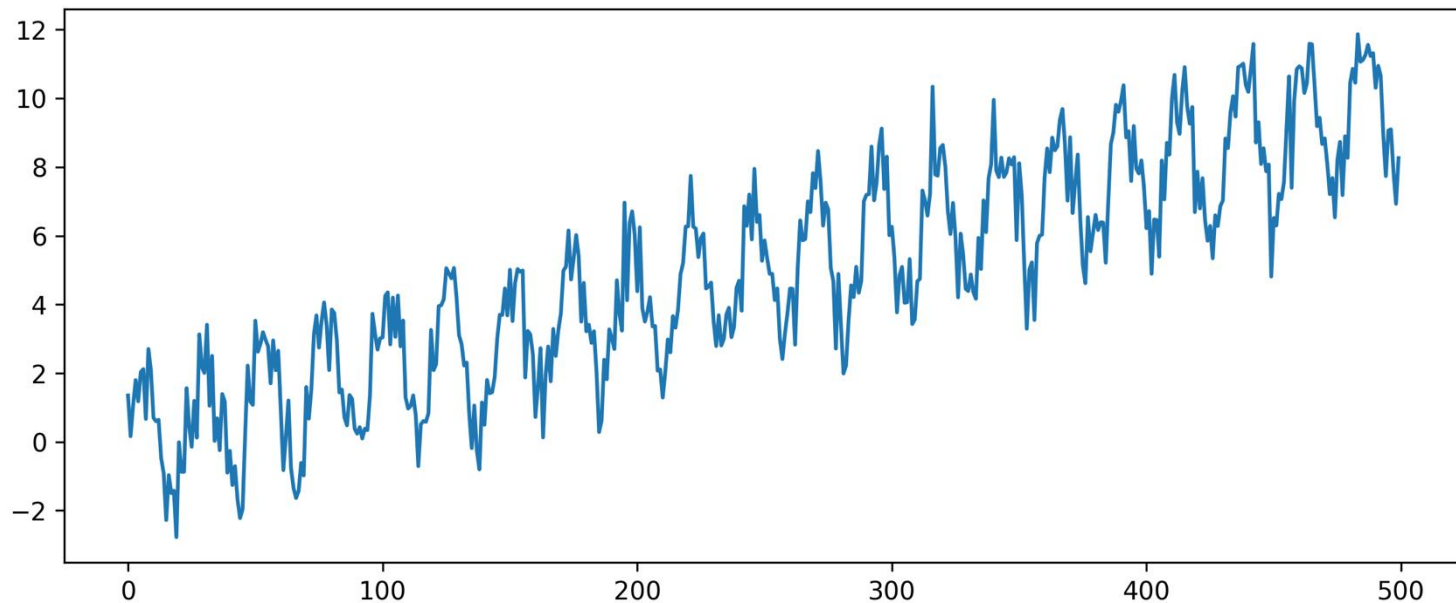
```
x = np.arange(N)
```

```
trend_true = 0.02 * x
```

```
season_true = 2 * np.sin(2*np.pi*x/period)
```

```
noise = np.random.normal(scale=0.8, size=N)
```

```
y = trend_true + season_true + noise
```



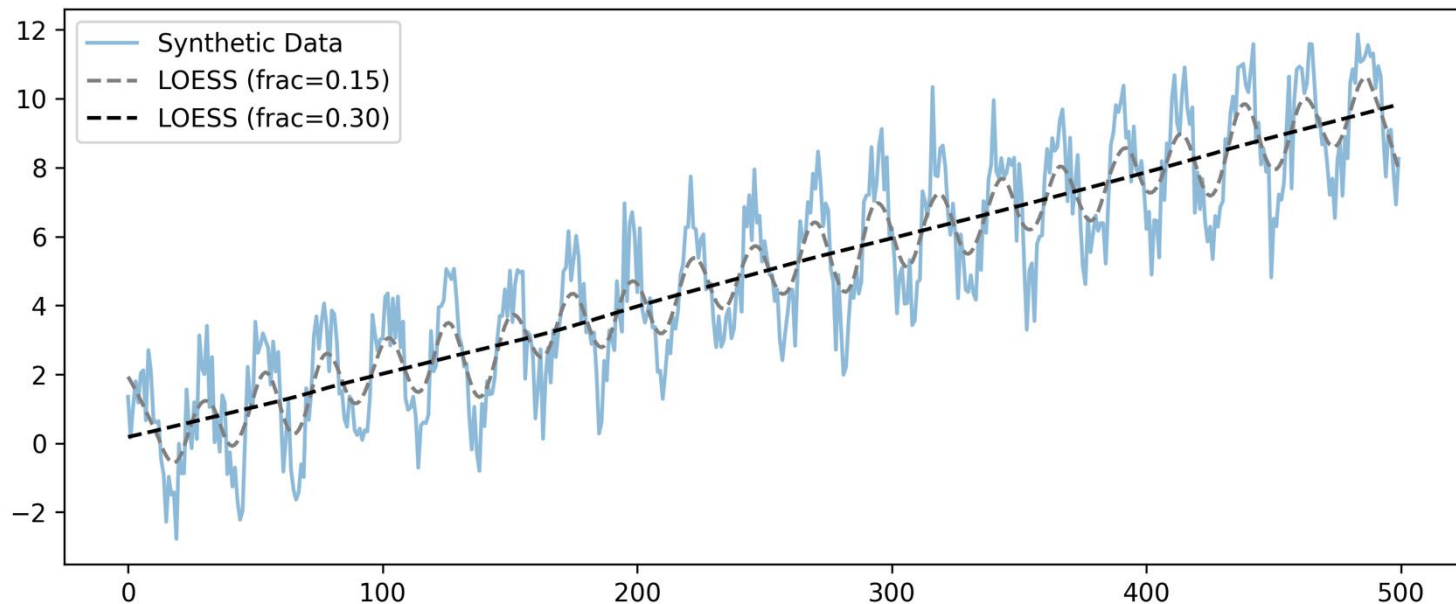
# LOESS in Python – synthetic data example

# 2) LOESS smoothing (local linear) for comparison

```
loess_curve_015 = lowess(y, x, frac=0.05, it=0,  
return_sorted=False)
```

```
loess_curve_030 = lowess(y, x, frac=0.3, it=0,  
return_sorted=False)
```

↑  
smaller **frac** → more local  
larger **frac** → smoother





Now we can  
get smoothed  
trend, how  
can we fully  
decompose  
time series to  
trend,  
periodicity,  
and noise?

# Seasonal-Trend Decomposition using LOESS (STL)

STL decomposes a time series into three components:

$$y_t = T_t + S_t + R_t$$

Where:

- $T_t$  :smooth long-term trend
- $S_t$  :repeating seasonal pattern (period  $m$ )
- $R_t$  :leftover noise or anomalies

LOESS enables STL's decomposition

- LOESS uses distance-based weights and local regression; STL iterates LOESS smoothers to separate trend, seasonal, and remainder components.

# STL algorithm in a nut shell

STL decomposes  $y_t$  into trend  $T_t$ , seasonality  $S_t$ , and remainder  $R_t$  using an **outer loop** (trend extraction / robust reweighting) that encloses an **inner loop** (seasonal extraction). Each smoothing step is done with **LOESS**, typically degree 0 or 1, over moving windows whose **odd integer lengths** you control.

$$y_t = T_t + S_t + R_t \text{ with } R_t = y_t - T_t - S_t$$

# STL Conceptual Steps

## **Outer Loop (Trend extraction)**

- Remove seasonal component
- Apply LOESS to get the trend  $T_t$

## **Inner Loop (Seasonal estimation)**

- Group observations by seasonal position (e.g., all January, all Mondays, all hours = 1, etc.)
- LOESS-smooth each seasonal subseries
- Combine to form the seasonal component  $S_t$

## **Final Step**

$$R_t = y_t - T_t - S_t$$

# Inner loop – estimate the seasonal component $S_t$

## 1. Create seasonal subseries

Split the series by seasonal position:  $\{y_t: t \equiv j \pmod{m}\}$  for  $j=1, \dots, m$ . Each subseries contains all observations at the same position within the cycle (e.g., all January, all “hour 5” values)

## 2. LOESS on each subseries (seasonal window).

For each  $j$ , smooth its subseries using LOESS with span = seasonal (deg 0 or 1). This gives preliminary seasonal estimates  $S_t^{(0)}$  at those positions.

## 3. Low-pass filter the seasonal (stabilize across cycle)

Apply a short **low-pass LOESS** (window = low\_pass) to the sequence of preliminary seasonal values to remove leakage of very low-frequency trend into the seasonal; this produces  $\tilde{S}_t$ . In the original algorithm, this replaces a more ad-hoc moving-average cascade.

## 4. Seasonal centering and reallocation.

STL enforces that the average seasonal within each full cycle is (approximately) zero by removing any overall level from  $\tilde{S}_t$  and adding it back to the trend  $T_t$ . This avoids duplication between  $S_t$  and  $T_t$ . Result:  $S_t \leftarrow \tilde{S}_t - \bar{\tilde{S}}_{cycle}$

# Outer loop – estimate the seasonal component $S_t$

## 1. Seasonally adjust and LOESS for trend (trend window).

Remove the seasonal:  $y_t^{(sa)} = y_t - S_t$ .

Smooth  $y_t^{(sa)}$  with LOESS (window = trend, deg 0 or 1) to get  $T_t$ . A larger trend window yields a slower-varying trend.

## 3. Backfitting / convergence.

Alternate **inner** (seasonal) and **outer** (trend + robustness) passes a few times (e.g., inner  $\approx 2$ , outer  $\approx 0-15$  in R's `stl`, user cannot configure it in Python `STL`), which is sufficient in practice

## 2. Compute remainder and (optional) robust weights.

$R_t = y_t - T_t - S_t$ . If **robust** is on, compute Tukey/bisquare weights  $b_t \in [0, 1]$  from standardized residuals and **multiply** them into subsequent LOESS weights. This down-weights outliers, yielding robust  $S_t, T_t$ . Repeat for a small number of **outer iterations**.

# STL in Python – synthetic data example

```
from statsmodels.tsa.seasonal import STL
```

```
# 4) STL decomposition
```

```
series = pd.Series(y, index=pd.RangeIndex(start=0,  
stop=N))
```

```
stl = STL(series, period=period, robust=True)
```

```
res = stl.fit()
```

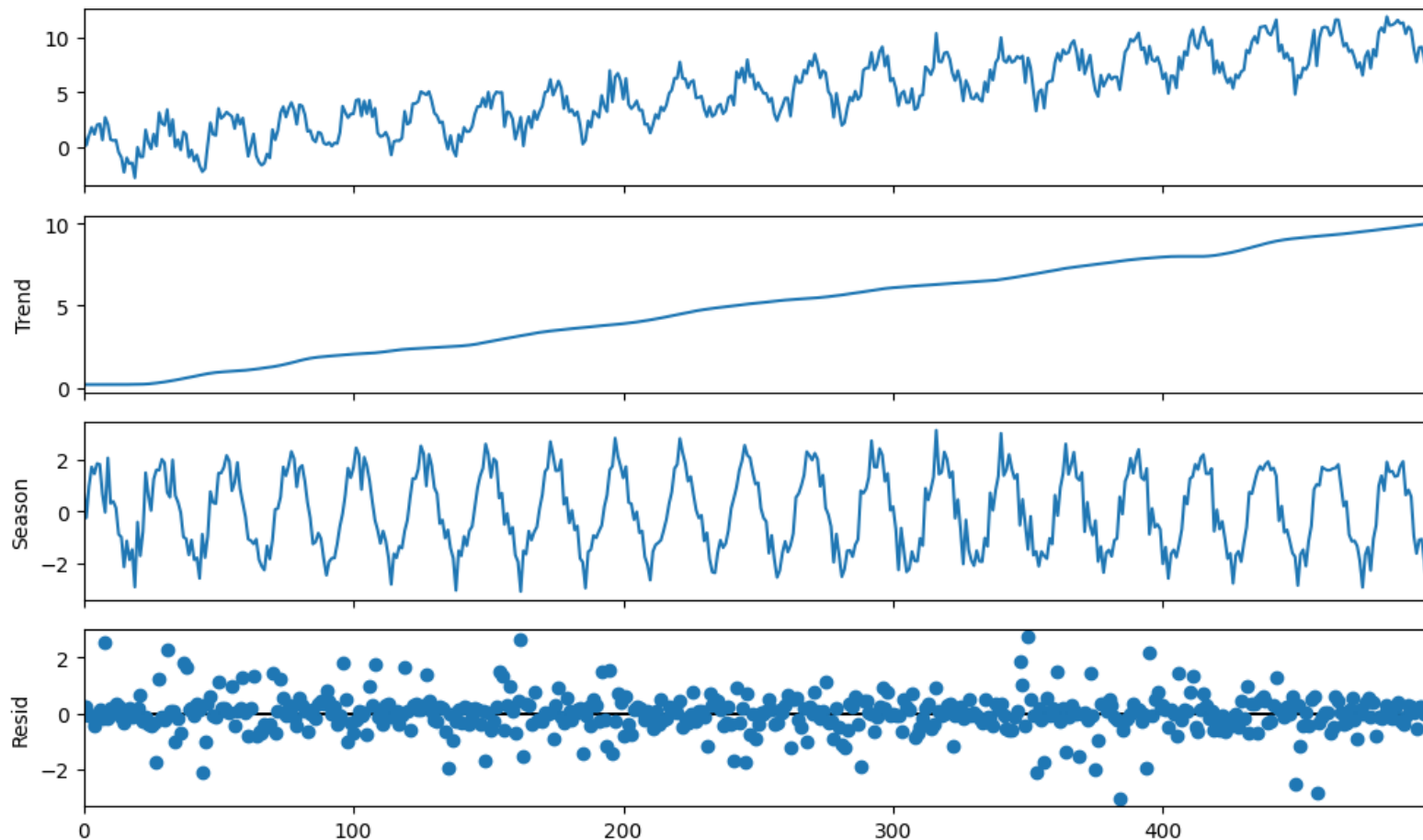
In STL algorithm, we must specify  
the period.

```
# 5) Plot STL components
```

```
fig = res.plot(); fig.set_size_inches(10,6)
```

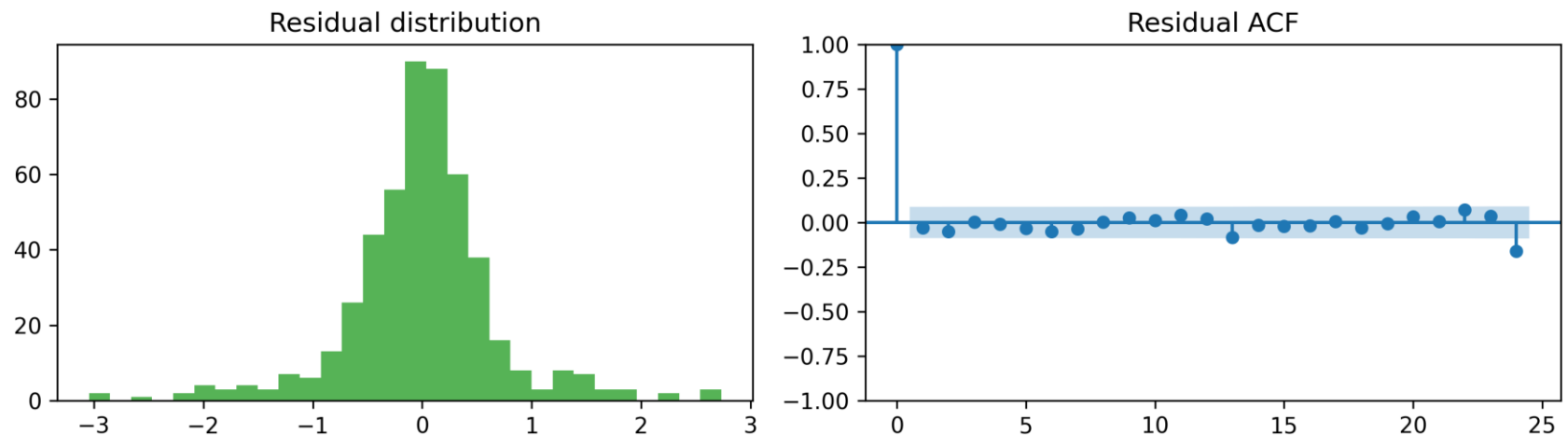
```
plt.tight_layout()
```

# STL in Python – synthetic data example



Now we can cleanly decompose the data into trend seasonality, and residuals!

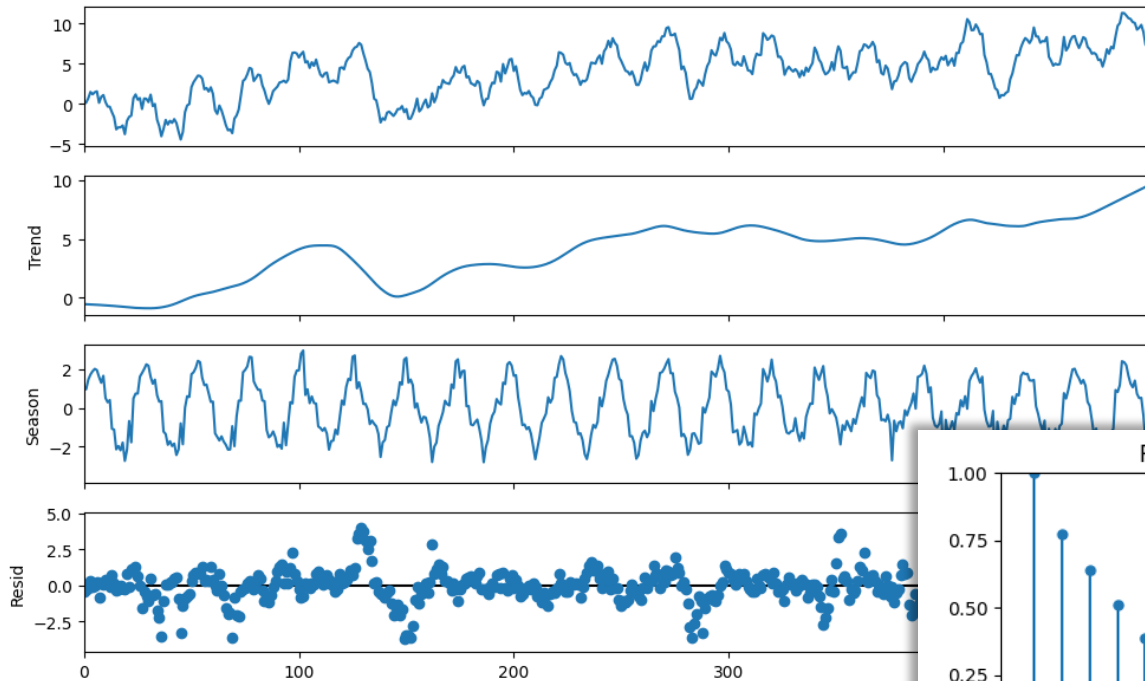
# Residual Diagnostics Matter in STL – teaser slides for auto-correlation



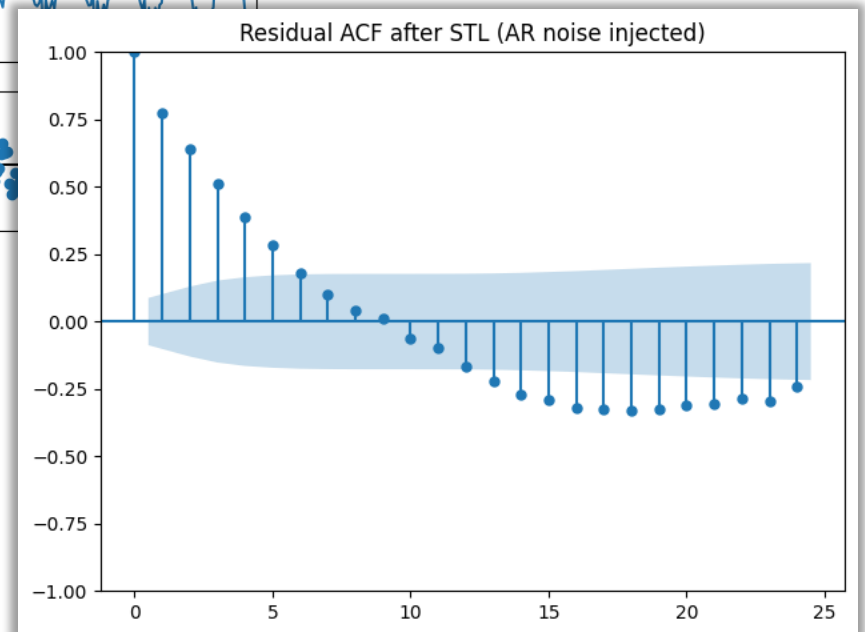
We would expect our residual term to be a white noise, i.e., similar to normal distribution, and with little auto-correlation.

**Unfortunately, that's not always the case...**

# An example with strong auto-correlation



When we plot the ACF, we get the following plot, indicating the residual term is strongly auto-correlated.



# Comparison

| Method                                            | What it does                                                                               | Best for              | Strengths                                                                | Weaknesses                                                                  |
|---------------------------------------------------|--------------------------------------------------------------------------------------------|-----------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>Simple Moving Average (SMA / rolling mean)</b> | Averages the last $k$ points with equal weights                                            | Very basic smoothing  | Fast, intuitive, removes high-frequency noise                            | Cannot capture curves; edge effects; does not model seasonality             |
| <b>LOWESS/LOESS</b>                               | Locally weighted polynomial regression                                                     | Nonlinear trends      | Flexible, captures curves, handles irregular data                        | Requires tuning span; does not automatically separate trend and seasonality |
| <b>STL (Seasonal-Trend LOESS)</b>                 | Iteratively separates <b>trend</b> , <b>seasonality</b> , and <b>remainder</b> using LOESS | Complex seasonal data | Handles changing seasonal amplitude, robust, interpretable decomposition | Requires setting seasonal & trend window lengths                            |

# Today's Learning Outcomes

1. Be able to fit a moving average to a time series
2. Be able to explain the properties of a moving average
3. Be able to explain the concept of STL